ECE 2031 Project Summary

Aiden Dowling, Devin Fromond, Kepler Boyce, Sophia Ho

Submitted
December 2, 2025

Introduction

This document summarizes the Math Peripheral our team created for SCOMP: a six-register block at I/O addresses 0x090–0x095 that executes signed add, subtract, multiply, divide, modulus, absolute value, negate, min, and max directly in hardware (Figure 1). Its purpose is to show future engineers what the peripheral does, how to drive it, and why we chose a deterministic CONTROL/STATUS protocol. We prioritized completing the arithmetic instruction set with flag coverage before building demos so that the hardware-software boundary stayed stable for the rest of the project's duration. Both MathStepper.asm and TriangleArea.asm validated that the final peripheral meets the requirements from our proposal, delivering single-cycle command latency and clear READY, OVERFLOW, and DIV_BY_ZERO status indicators.

Opcode	Function	Operation
0x0	NOP	–
0x1	ADD	A + B
0x2	SUB	A – B
0x3	MUL	A x B
0x4	DIV	A / B
0x5	MOD	A % B
0x6	ABS	| A|
0x7	NEG	– A
0x8	MIN	MIN(A, B)
0x9	MAX	MAX(A, B)

Figure 1. Opcode table for arithmetic operations.

Device Functionality

Programmers interact with six locations: CONTROL selects the opcode, OPERAND_A and OPERAND_B capture inputs, RESULT_LO and RESULT_HI return the output, and STATUS exposes READY plus the overflow and divide-by-zero bits (Figure 2). A full transaction is five instructions: write both operands, write CONTROL with the opcode, then read RESULT_LO/RESULT_HI. STATUS updates in the same cycle as CONTROL, so code can immediately branch on flags without waiting or issuing dummy reads. Multiplication returns a 32-bit product split across LO and HI, while divide/modulus return quotient in LO and remainder in HI so software can use whichever word it needs without another command. The MathStepper.asm demo runs through every opcode and mirrors STATUS on LEDs, giving users visual confirmation of how each flag behaves. TriangleArea.asm chains operational results to compute (base × height)/2 with no extra logic, providing a practical math demo that collapses to a basic simple read/write rhythm. Because the peripheral never asserts wait states, it integrates neatly into existing SCOMP programs that already expect memory-mapped I/O timing.

Offset	Default Addr	Name	Access
+0	0x090	CONTROL	W
+1	0x091	OPERAND_A	W
+2	0x092	OPERAND_B	W
+3	0x093	RESULT_LO	R
+4	0x094	RESULT_HI	R
+5	0x095	STATUS	R

Figure 2. Register map for arithmetic operations.

Design Decisions

We implemented the peripheral as a synchronous process that latches operands when io_wr asserts, executes the requested operation combinationally, and rewrites RESULT and STATUS before the CONTROL pulse completes. This approach guarantees that the peripheral always reads coherent data without needing a BUSY flag. Signed overflow detection for add/sub compares operand and result sign bits; multiply widens internally to 32 bits and only raises OVERFLOW when RESULT_HI is not a sign extension, giving programmers confidence in 16-bit products. Divide and modulus share one datapath, and the quotient/remainder dual return avoids duplicate hardware while still serving both use cases. We also chose to keep READY high whenever the peripheral is idle, so polling loops only need one conditional branch. Development-wise, we used MathStepper.asm as an interactive verification tool, then built TriangleArea.asm to demonstrate a simple, practical use-case. These iterations confirmed that we did not need intermediate states or software-based exception handling and allowed us the opportunity to develop demos tailored for two distinct purposes.

Conclusion

The finished peripheral offloads the tedious parts of signed arithmetic from SCOMP, exposes a predictable polling interface, and incorporates two small demos that future engineers can reuse to understand and test the peripheral. In retrospect, this project may have been simpler if we solidified the register usages earlier and trimmed additional complexities (such as switch debouncing) to keep every decision focused on math behavior. Future teams extending this work should preserve the simultaneous quotient/remainder outputs and the immediate flag updates, since those traits made the peripheral easy to integrate. Additionally, MathStepper.asm should not be utilized to understand the math peripheral; TriangleArea.asm is more user-friendly whereas MathStepper.asm is more output-driven.